

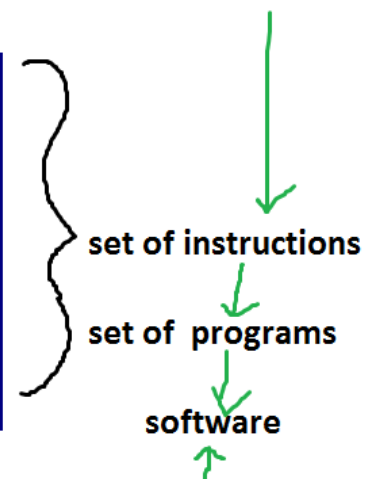
INTRODUCTION TO C

C is a

1. High level / middle level programming language.
2. C is a compiler based programming language.
3. C follows procedure oriented programming structure [POP's]

c / c++ / java / py / .net - programming languages

```
/* source code / source program */
#include<stdio.h>
#include<conio.h>
void main()
{
    int i; /* variable */
    clrscr();
    for(i=1;i<=10000;i++)
    printf("%d ",i);
}
```



software engineer / programmer / developers / techies

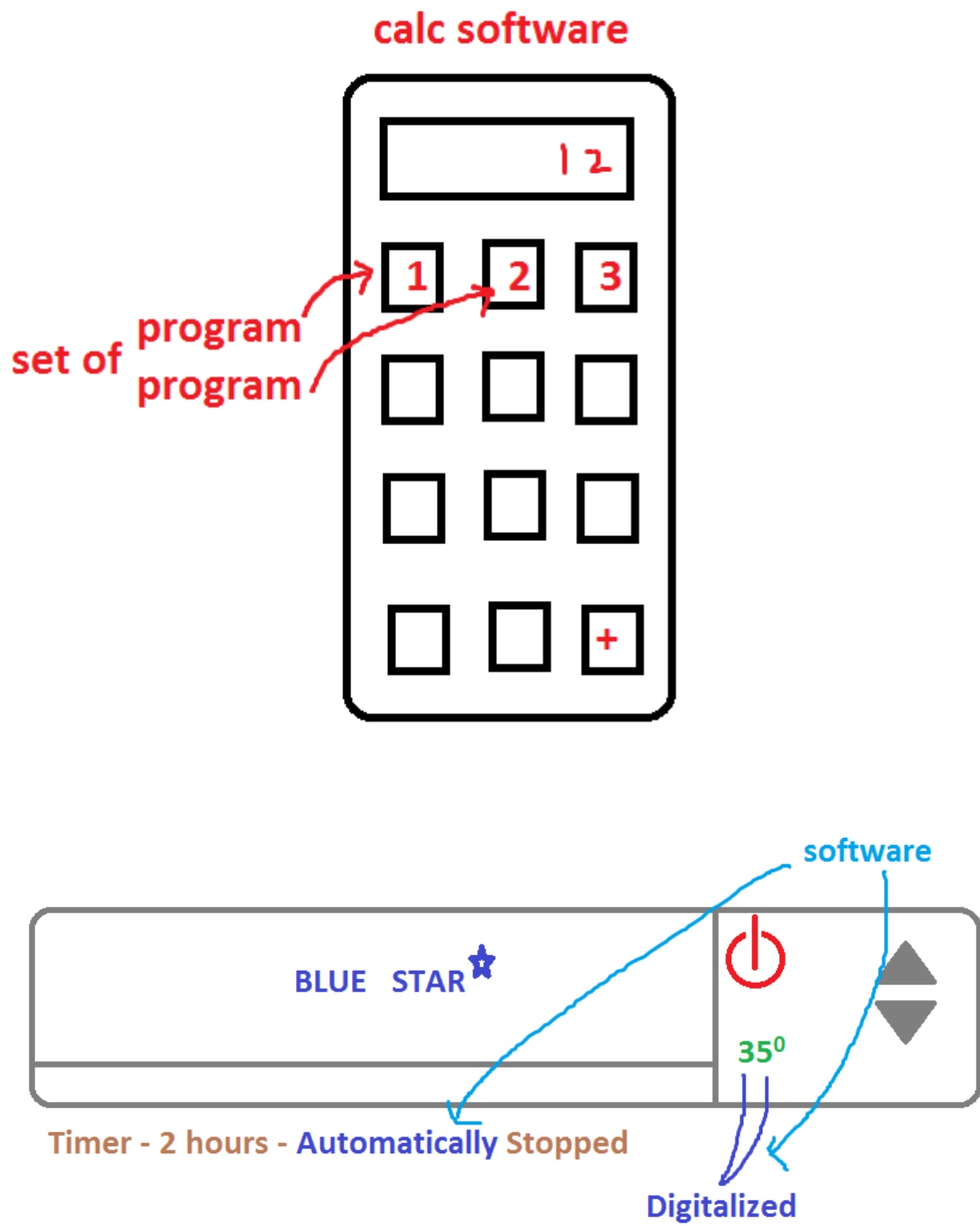
wipro / tcs / infosys / tech / microsoft / oracle - software companies

What is a program?

Set of instructions is called program.

What is a software?

Set of programs is called software / As per IT industry software is a digitalized and automated process.



We are having 2 type of software.

1. System software

Eg: operating system, translators, device drivers

2. Application software

Eg: Whatsapp, insta, phonepe,....

What is a language?

Generally the languages are used to communicate with others. We are using the human languages like telugu, English, hindi, Marathi etc are used to communicate with humans. But by using these human languages we can't communicate with machines.

For that we are using the **computer programming languages** like C / C++ / Java / Py / .net etc, which allows to write the programs [software] to communicate with devices.

Basically the computer languages divided into 3 types.

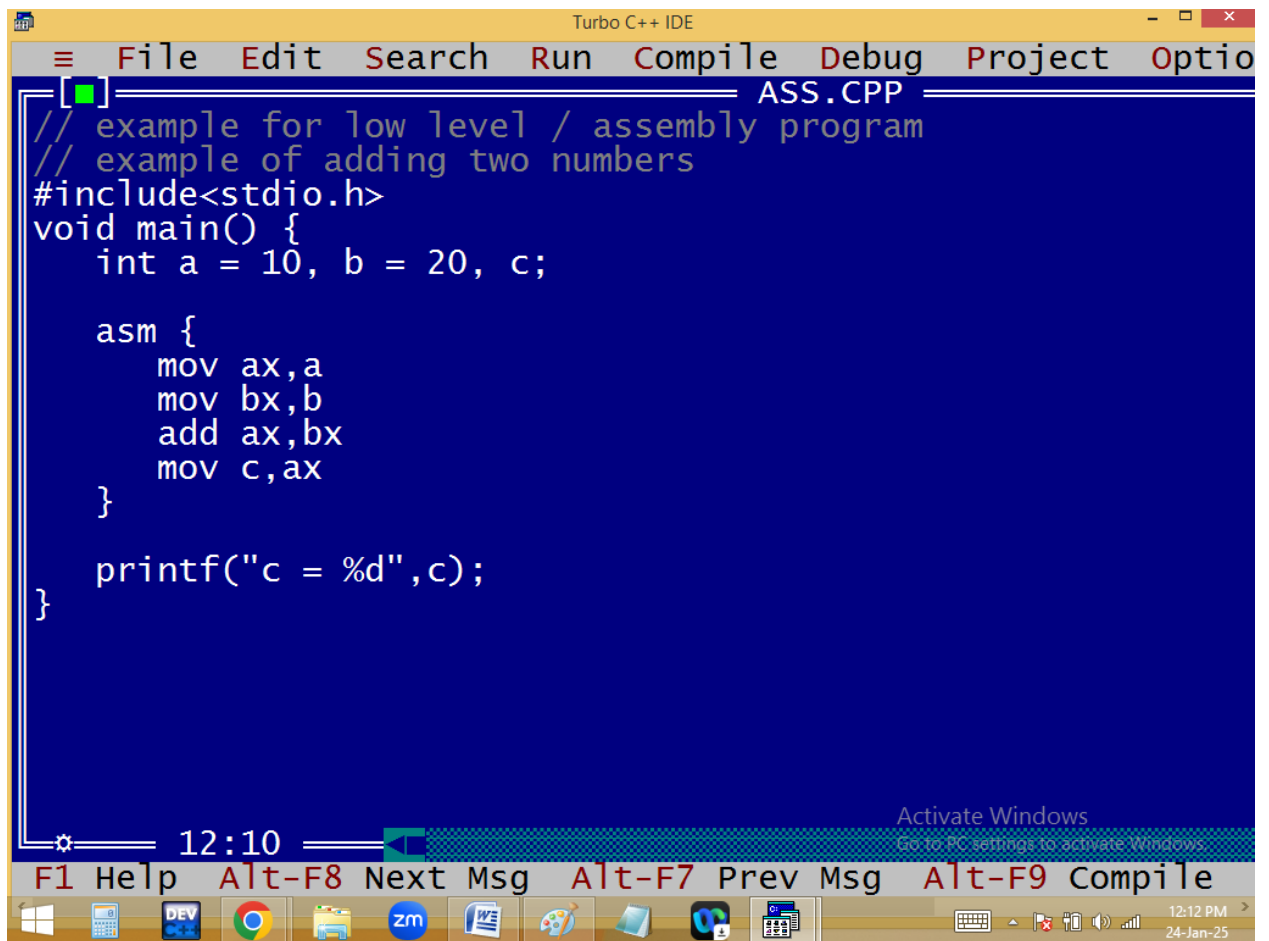
1. **Machine language**: Created with binary code

Eg: 1000111

2. **Low level/assembly language**: Created with English shortcuts called **MNEMONICS**.

Eg: sub, add,

Example:



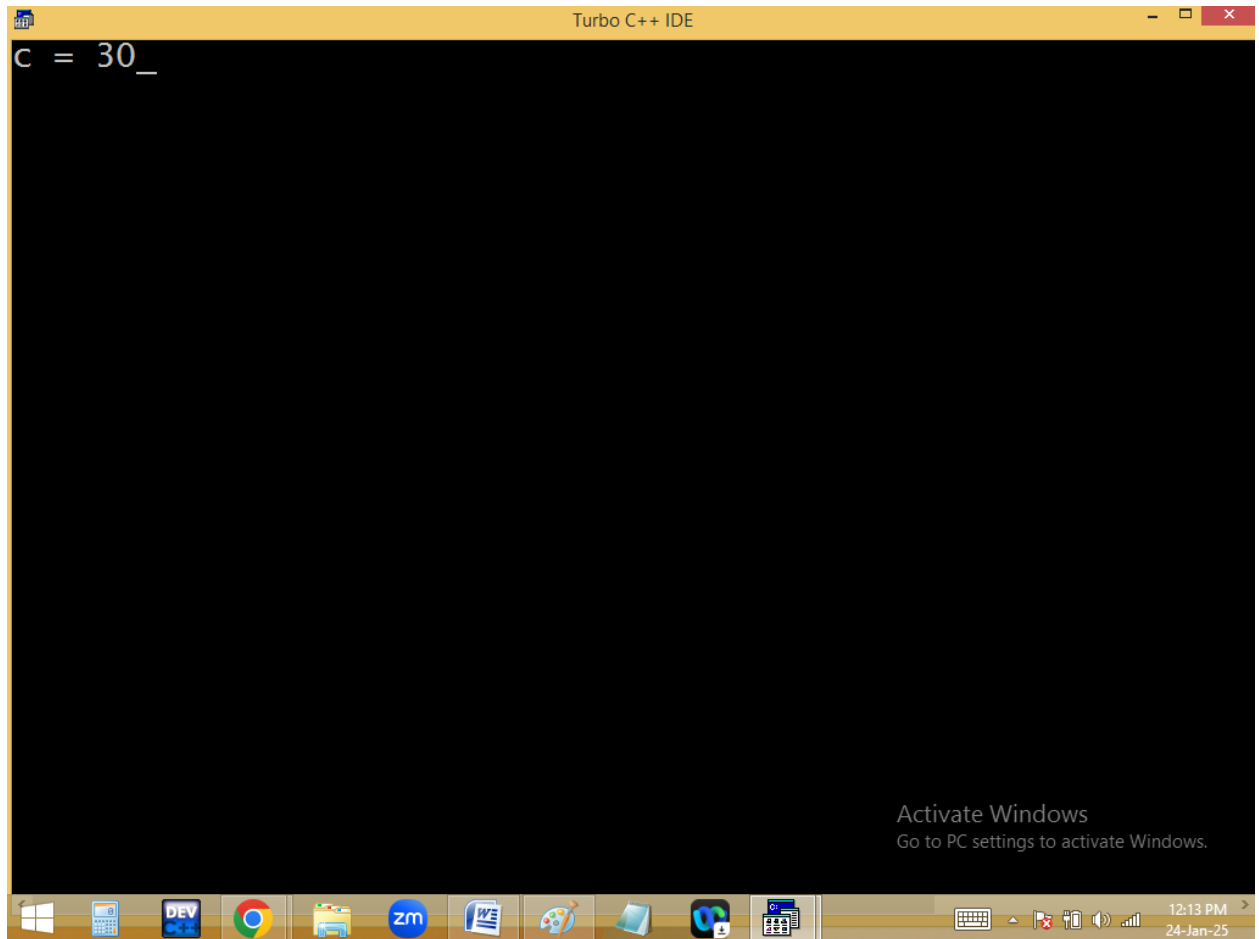
The image shows a screenshot of the Turbo C++ IDE. The title bar reads "Turbo C++ IDE". The menu bar includes "File", "Edit", "Search", "Run", "Compile", "Debug", "Project", and "Options". The file name "ASS.CPP" is displayed in the top right corner of the editor. The code in the editor is as follows:

```
// example for low level / assembly program
// example of adding two numbers
#include<stdio.h>
void main() {
    int a = 10, b = 20, c;

    asm {
        mov ax,a
        mov bx,b
        add ax,bx
        mov c,ax
    }

    printf("c = %d",c);
}
```

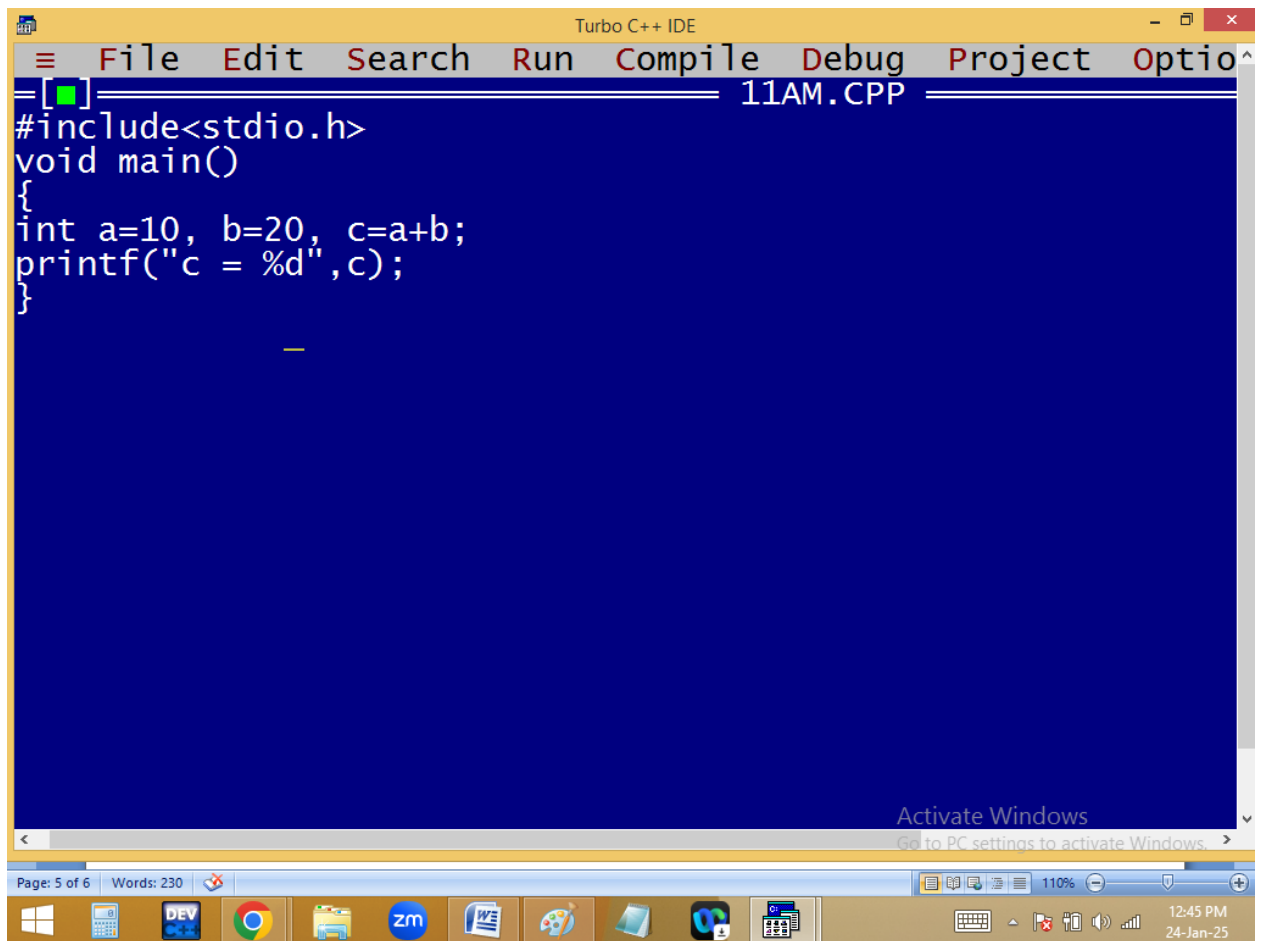
The status bar at the bottom shows the time "12:10" and a system tray with various icons. A taskbar at the very bottom contains icons for Windows, a calculator, a development environment, Google Chrome, a file explorer, a Zoom icon, a web browser, a game controller, a folder, a music note, and a calendar. The system clock in the bottom right corner shows "12:12 PM" and "24-Jan-25".

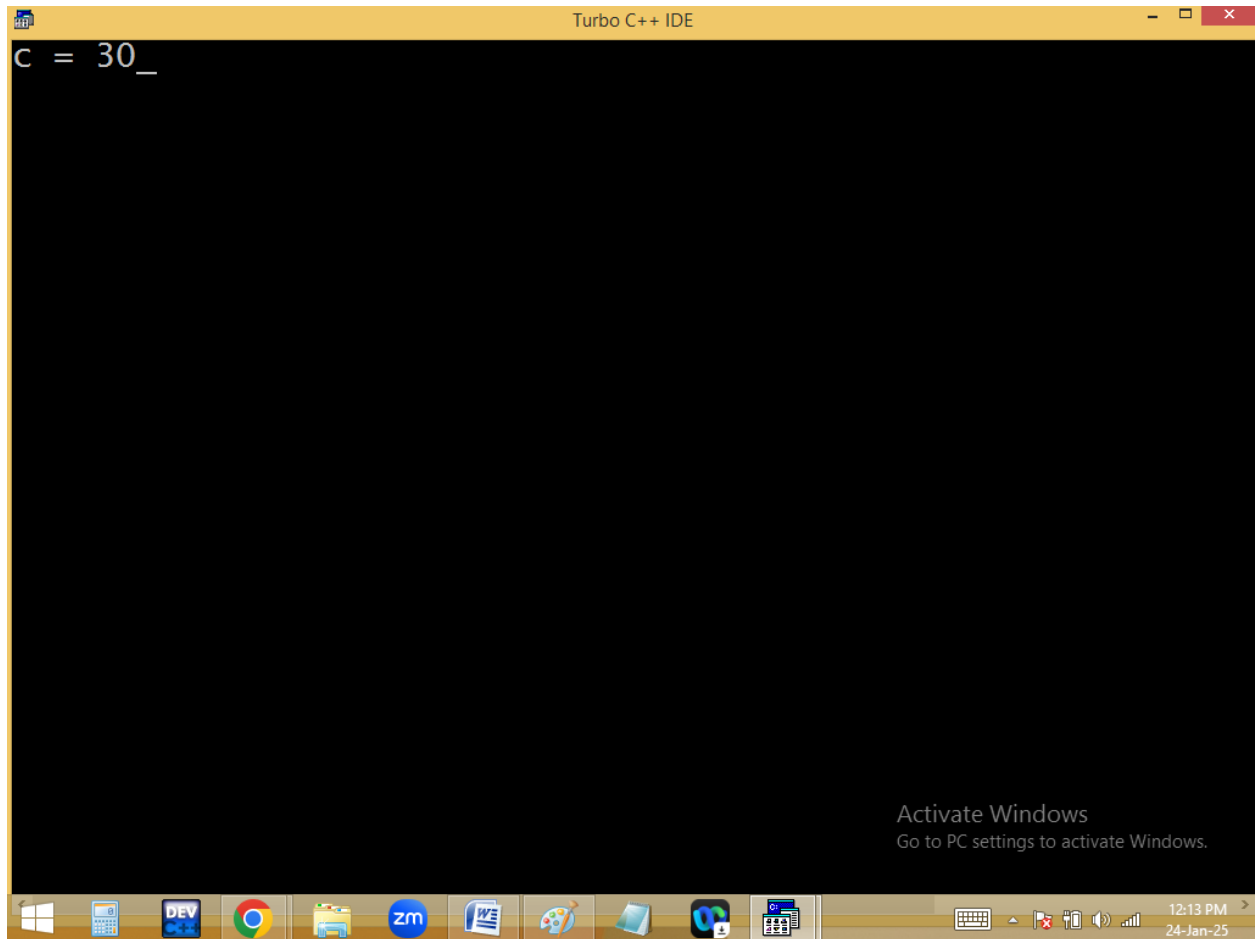


3. **High level language**: Created with simple English.

Eg: good morning, subject, addition,

Example:





C is a high level language with low level features. Hence it is also called middle level language.

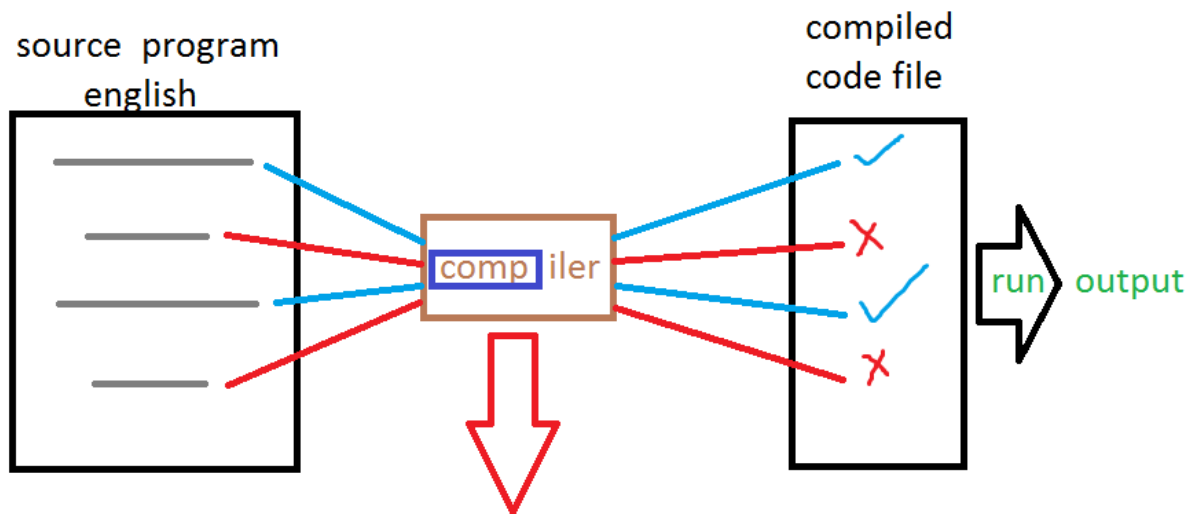
C high level features used to develop application software and low features used to develop system software. Hence C is a multi-purpose programming language.

What is a translator?



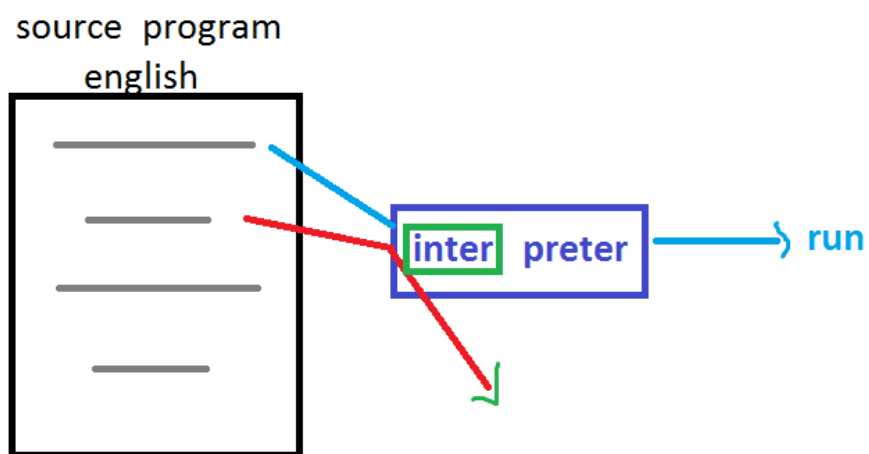
Compiler and interpreter used to convert our high level programs to binary code.

Compiler converts the total source code into binary code at once by leaving error lines.



After **completion** of total program it shows the errors

Interpreter converts **line by line**.



showing error in 2nd line [**middle**] and not checking remaining lines

Assembler is used to convert low level programs to binary code.

Assembler working style is similar to compiler.

In c we are using compiler. Hence it is a compiler based programming language.

In Java / .Net / Py we are using both compiler and interpreter. Hence they are called compiler based interpreted languages.

What is called programming paradigm?

Every programming language is having certain rules and regulations with a particular programming architecture. As programmer we should have to follow these rules and structure, which is technically called programming paradigm.

Before C language, the languages are using monolithic programming structure.

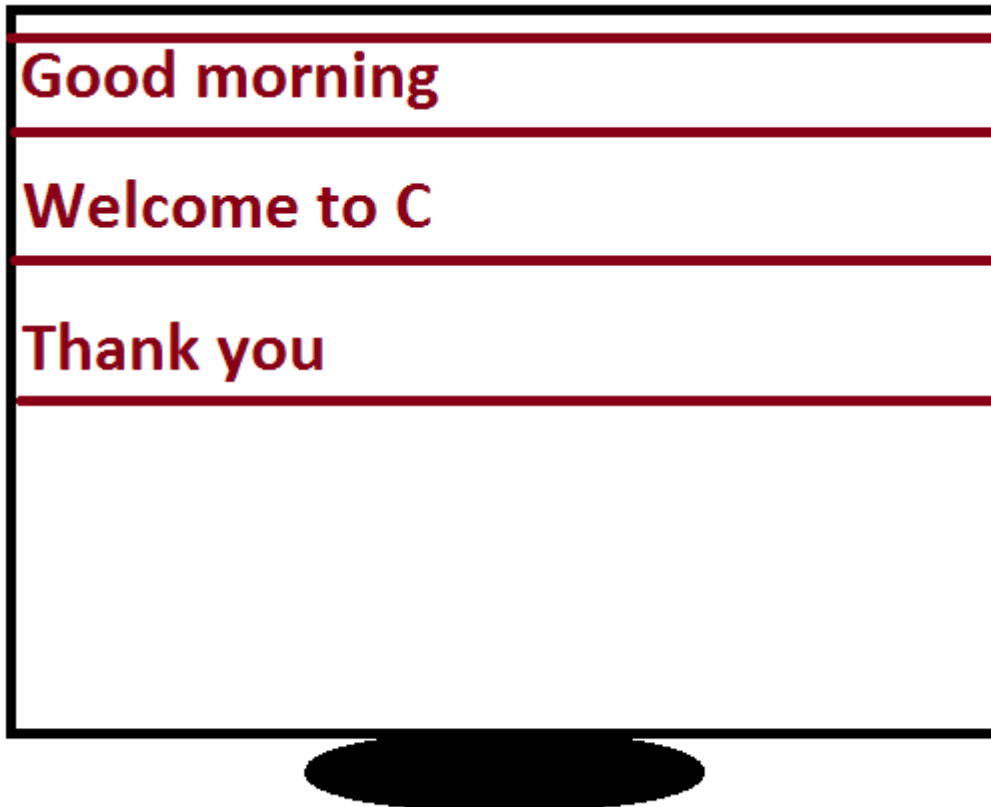
C is using Procedure oriented programming Structure

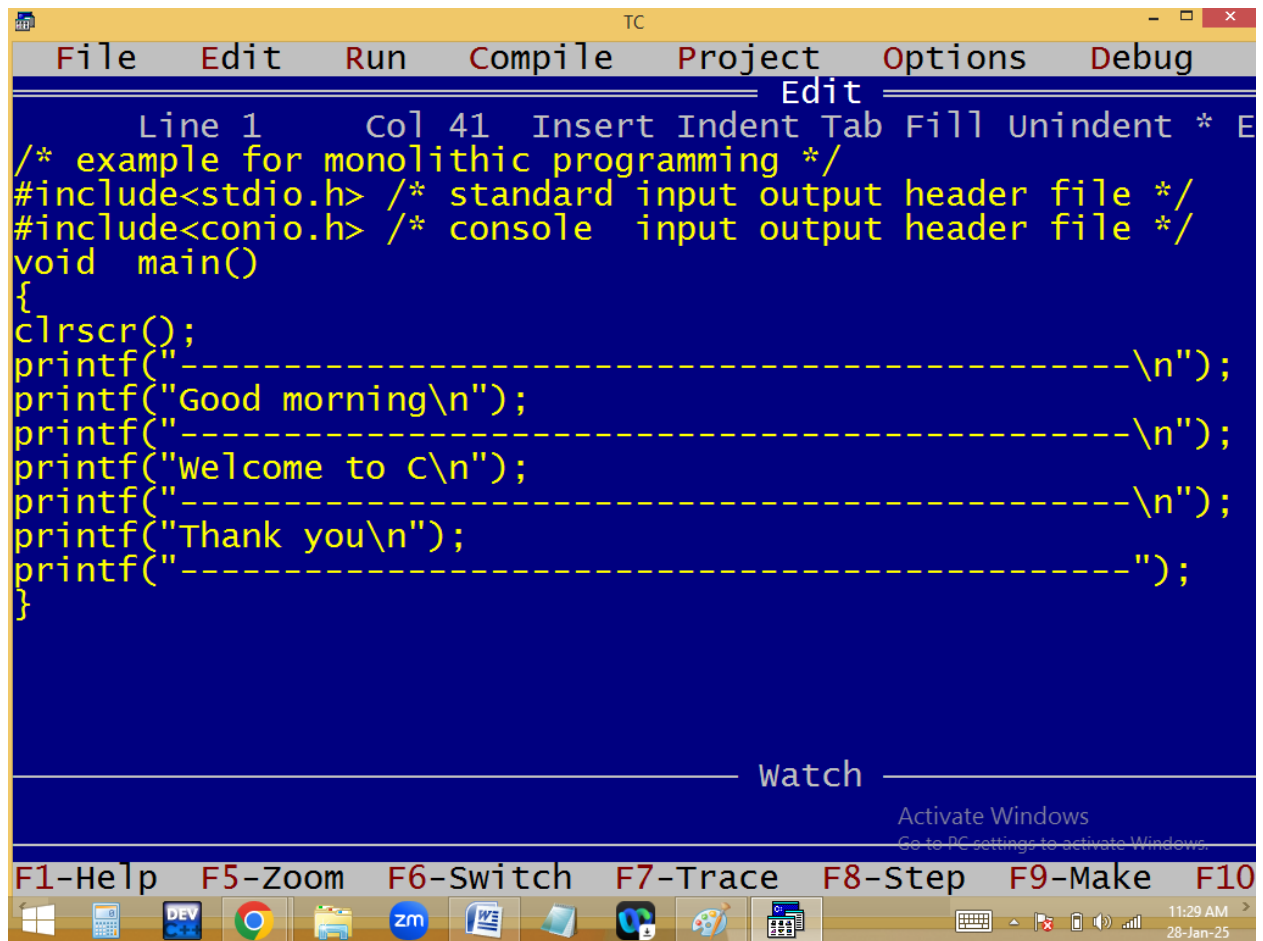
C++ and Python are using Procedure oriented and Object oriented programming Structures. Hence they are also called multi paradigm programming languages.

Before C language the languages are using monolithic programming structure, where the entire application

designed by using of single program. Due to this it is very difficult to find the errors, takes more memory and performance is low.

Example for monolithic programming:

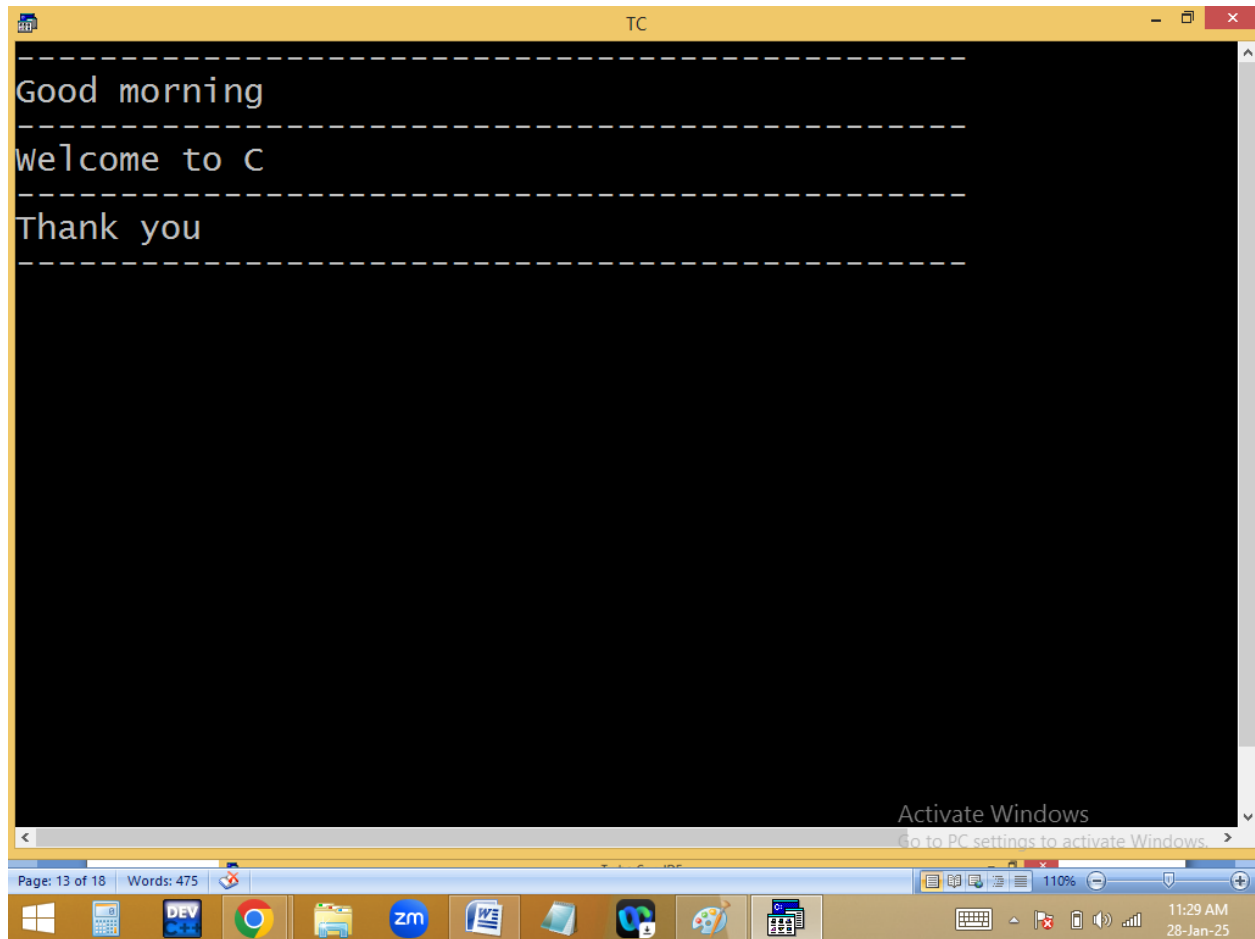




The image shows a screenshot of the Turbo C++ (TC) IDE. The window title is "TC". The menu bar includes "File", "Edit", "Run", "Compile", "Project", "Options", and "Debug". The "Edit" menu is open, showing options: "Line 1", "Col 41", "Insert", "Indent", "Tab", "Fill", "Unindent", and "* E". The main text area has a dark blue background with yellow text. It contains a C program for monolithic programming. The code is as follows:

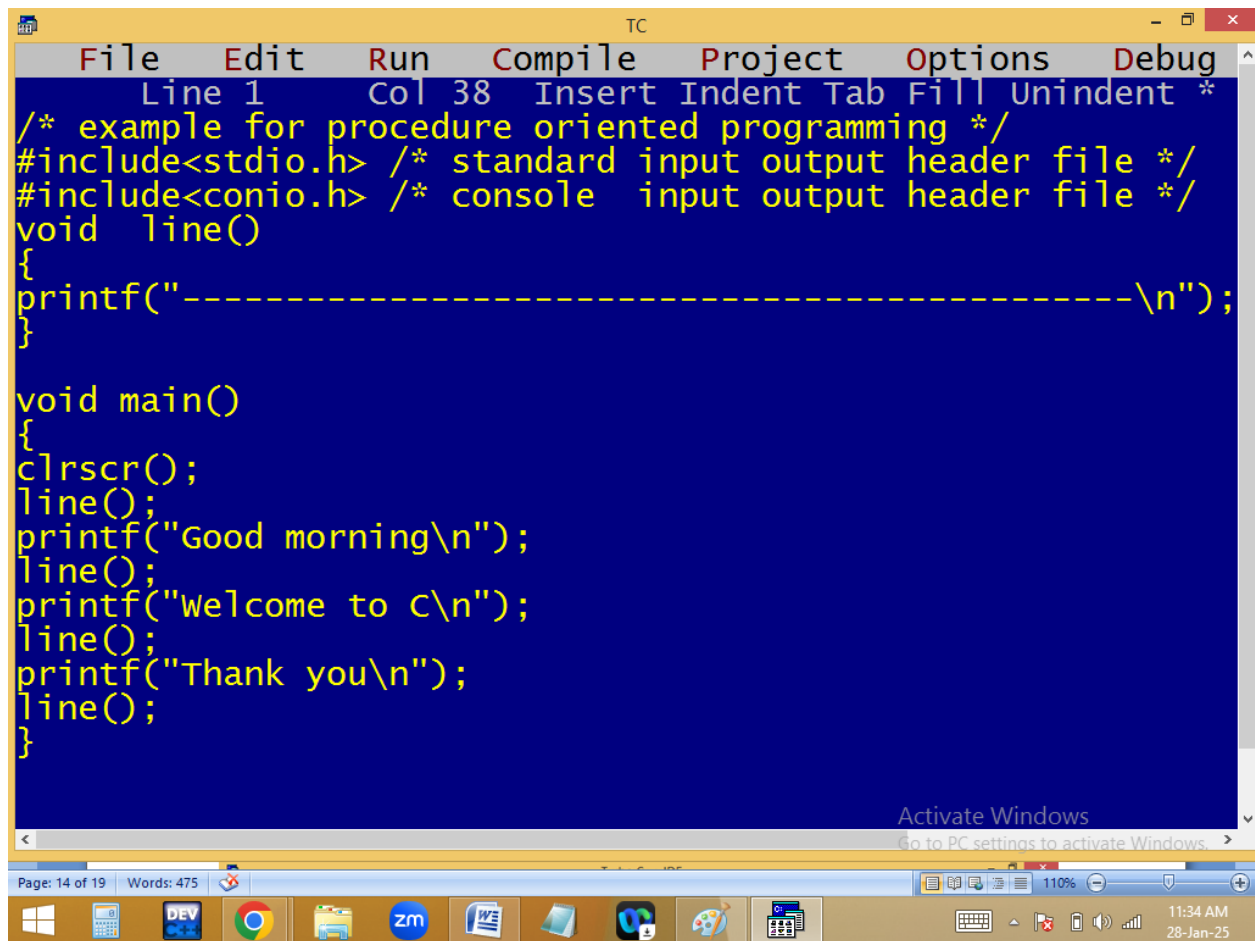
```
/* example for monolithic programming */
#include<stdio.h> /* standard input output header file */
#include<conio.h> /* console input output header file */
void main()
{
    clrscr();
    printf("-----\n");
    printf("Good morning\n");
    printf("-----\n");
    printf("Welcome to C\n");
    printf("-----\n");
    printf("Thank you\n");
    printf("-----");
}
```

Below the code area is a "Watch" window, which is currently empty. At the bottom of the IDE, there is a status bar with function key shortcuts: "F1-Help", "F5-Zoom", "F6-Switch", "F7-Trace", "F8-Step", "F9-Make", and "F10". The Windows taskbar is visible at the very bottom, showing icons for Windows, File Explorer, Dev-C++, Google Chrome, and other applications. The system clock in the bottom right corner shows "11:29 AM" and "28-Jan-25".



To avoid the problems in monolithic programming, they have introduced the procedure oriented programming structure [POP's]. in POP's a big program divided into several small sub programs / sub routines / functions / procedures / modules / structures.

Example:



The image shows a screenshot of a Turbo C++ (TC) IDE window. The title bar reads "TC". The menu bar includes "File", "Edit", "Run", "Compile", "Project", "Options", and "Debug". The status bar at the top indicates "Line 1", "Col 38", and lists actions: "Insert", "Indent", "Tab", "Fill", "Unindent", and "*". The main editing area has a dark blue background with yellow text. The code is as follows:

```
/* example for procedure oriented programming */
#include<stdio.h> /* standard input output header file */
#include<conio.h> /* console input output header file */
void line()
{
printf("-----\n");
}

void main()
{
clrscr();
line();
printf("Good morning\n");
line();
printf("Welcome to C\n");
line();
printf("Thank you\n");
line();
}
```

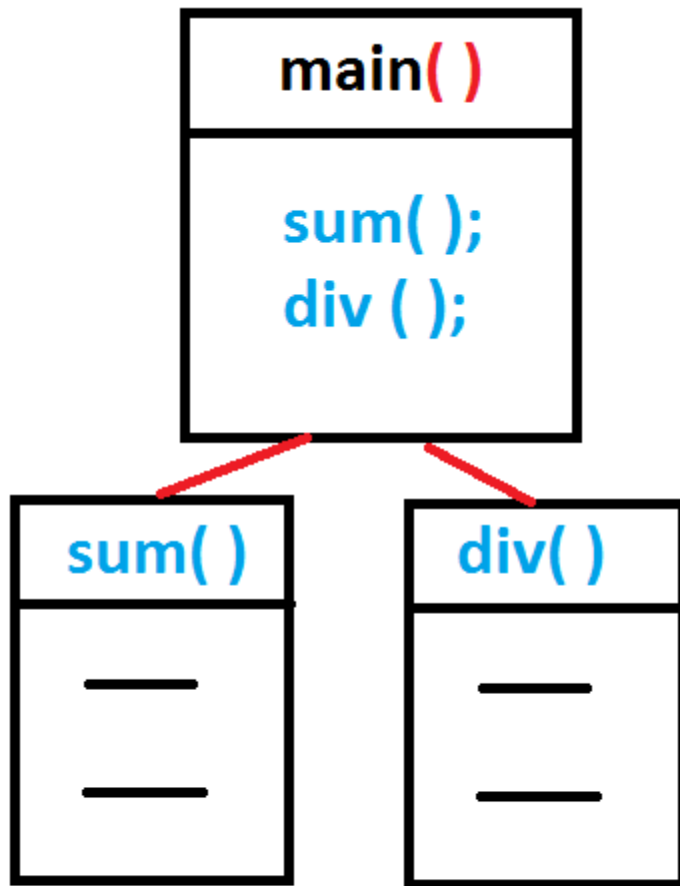
At the bottom right of the IDE window, there is a watermark that says "Activate Windows" and "Go to PC settings to activate Windows." Below the IDE window is the Windows taskbar. It shows the Start button, taskbar search, and several pinned applications: Calculator, DEV C++, Google Chrome, File Explorer, Zoom, Word, and a folder icon. The system tray on the right shows the date and time as "11:34 AM" and "28-Jan-25".

```
-----  
Good morning  
-----  
Welcome to C  
-----  
Thank you  
-----
```

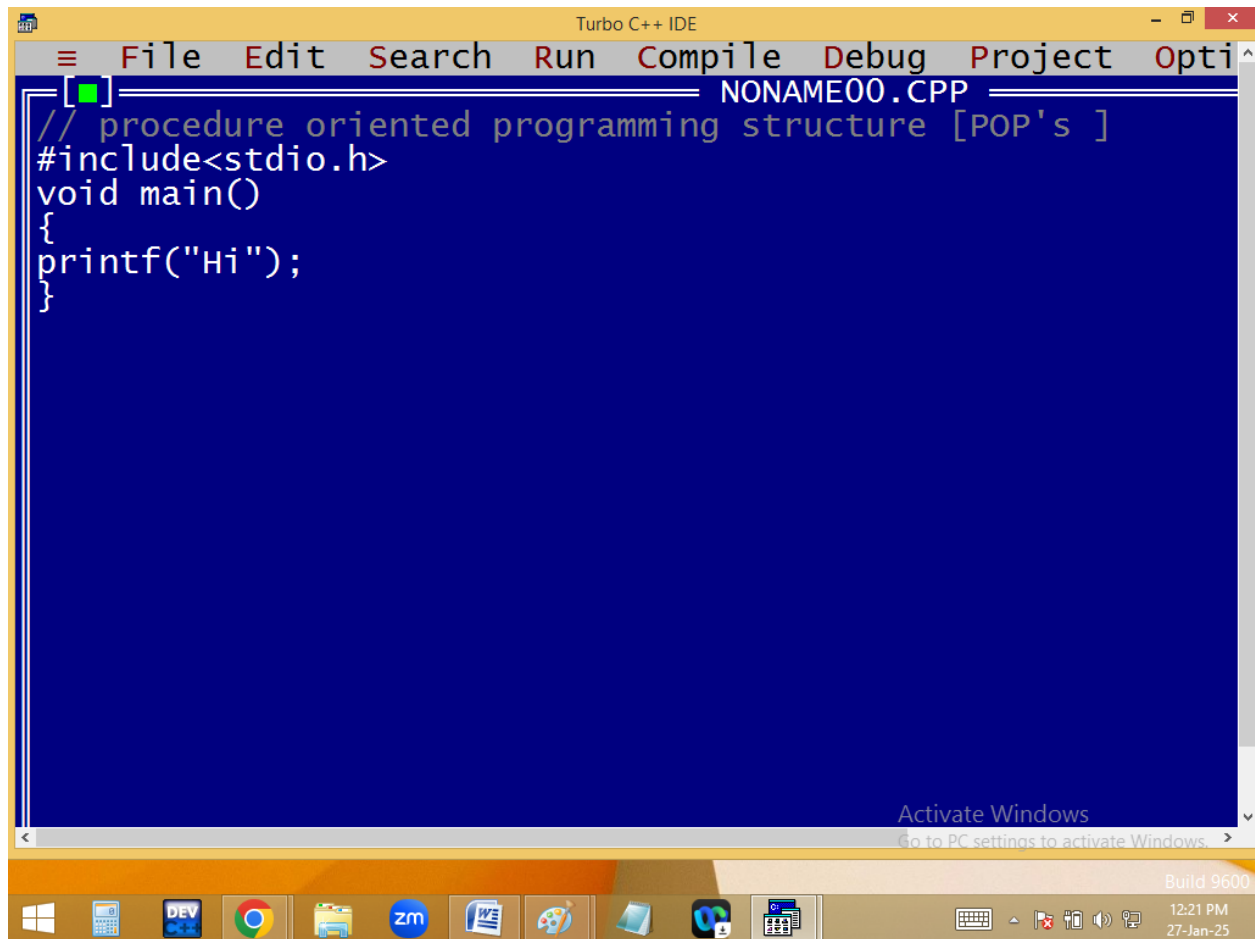
Advantages of POP:

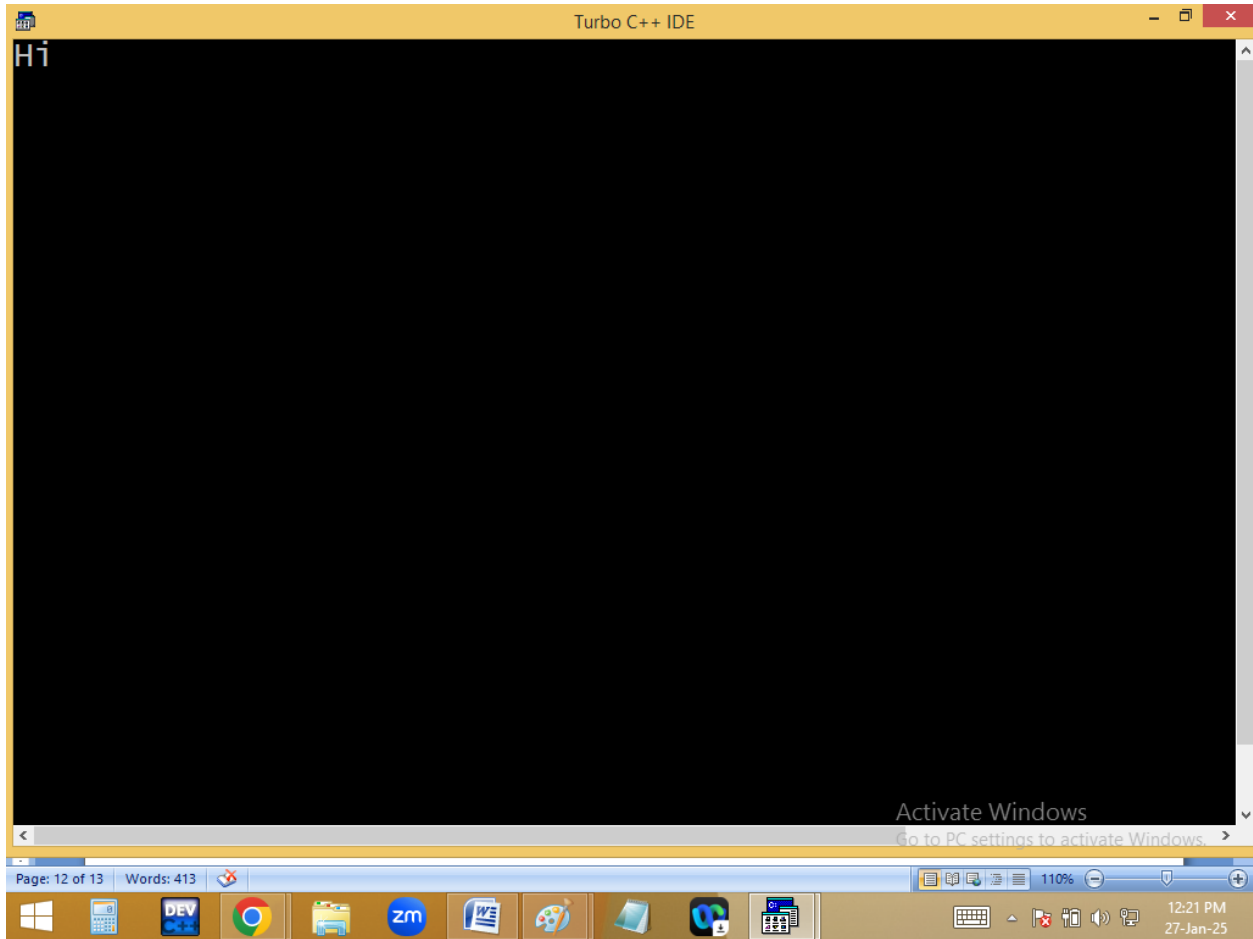
- 1. Modularity:** Dividing big program into small sub programs as per the project requirement.
- 2. Simplicity:** Easy to read and understand.
- 3. Reusability:** Write once, use many times.
- 4. Efficiency:** performance is high.

POPs structure:



sub prog's / functions/ procedure





Turbo C++ IDE

File Edit Search Run Compile Debug Project Opti

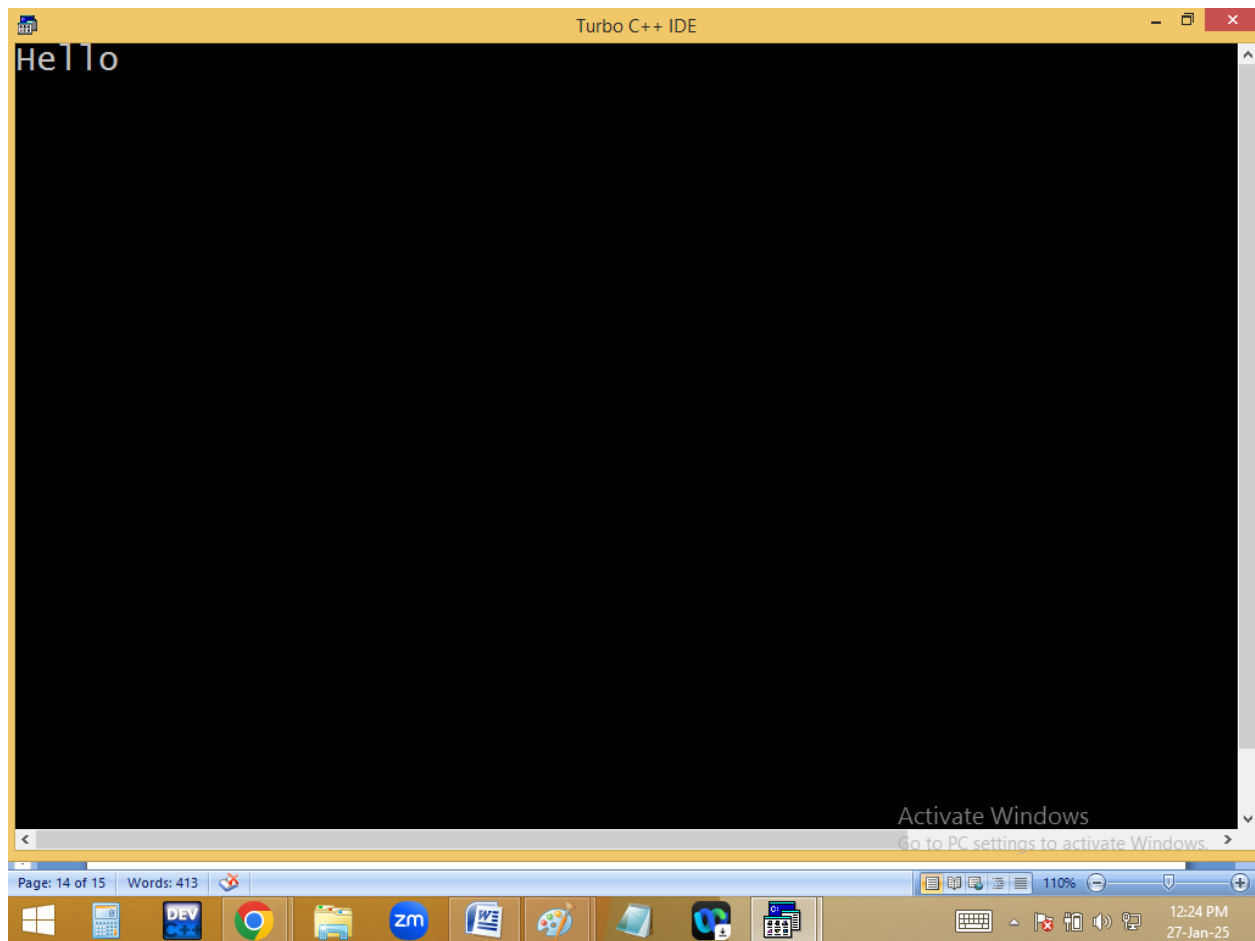
NONAME00.CPP

```
// Object oriented programming structure [POP's ]
#include<stdio.h>
#include<conio.h>
class test
{
public:
void show()
{
printf("Hello");
};
};
void main()
{
test t; // object
clrscr();
t.show();
}
```

Activate Windows
Go to PC settings to activate Windows.

Page: 13 of 14 Words: 413 110%

12:23 PM
27-Jan-25



Java and .net are using object oriented programming structure.

Modularity:

Simplicity

Reusability: Write once, use many times.

Efficiency: performance high.

```

/* Eng-source pro */_
#include<stdio.h>
void main()
{
printf("Good morning");
}

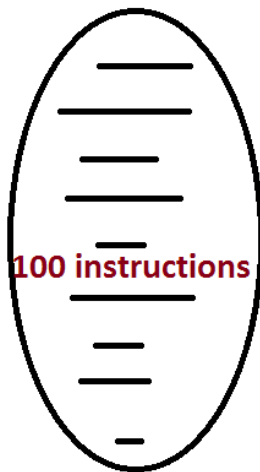
```



Translators
1. **compiler**
2. **interpreter**
3. **assembler**

1. **Checking errors**
 1. compile time
 2. runtime
 3. warnings
2. **eng to binary**

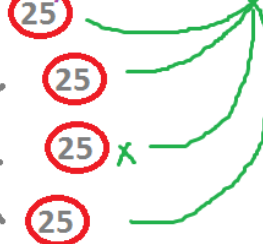
program-source code
english



monolithic programming

compiler
Error checking
eng to binary

subprog/subroutines/procedures/functions /
structure / modules



procedure oriented programming
structure - POP's

- dis: monolithic**
1. **diff find error**
 2. **takes more memo**
 3. **performance low**
 4. **no reusability**

OOP's – Object Oriented Programming Structure

1. **Class**
2. **Object**
3. **Data Hiding**
4. **Encapsulation**
5. **Inheritance**

6. Polymorphism – many – forms / shapes / kinds

7. Abstraction